

Game Network Protocols Production Design for Selecting and

Operating Transport and Messaging Date: 2025-08-10 UTC

Game Network Protocols Production Design for Selecting and
Operating Transport and Messaging Date: 2025-08-10 UTC

One Pager Summary Goal Define a secure, low-latency, and resilient networking stack for real-time and turn-based games. Establish transports, message semantics, reliability classes, congestion control, NAT traversal, and security controls that scale from thousands to millions of concurrent players across regions.

Key Outcomes

Consistent input-to-action under one hundred milliseconds p95 in-region for fast-action modes and sub-second for casual modes. Predictable reliability using multiple channels reliable ordered, reliable unordered, unreliable sequenced, fire-and-forget. Loss and jitter tolerance through pacing, selective retransmit, and interpolation on clients. Zero trust posture with encryption, token binding, rate limits, and DDoS-aware relays. Operational excellence with clear SLOs, canaries, rollback, deep observability, and cost-aware scaling.

Primary Assumptions

Global player base across North America, Europe, Asia Pacific with regional cells. Modes include fast-action real-time, cooperative PvE, and asynchronous social features. Clients on PC, console, mobile; some restrictive networks require websocket fallback. Authoritative servers run simulations; voice is handled by a separate media path when needed.

Functional Requirements

Transports Support UDP-based protocol for data plane with QUIC datagrams for low-latency unreliable streams and ordered control streams. Provide websocket fallback. Support TURN relays for restricted networks. Message Model Multiple reliability classes. Message framing with length prefix, type id, sequence id, timestamp, and optional ack bits. Fragmentation and reassembly for large payloads within policy limits. Matchmaking and Session QoS probing and tokenized join. Session resume and fast reconnect with state warmup. Clock and Tick Client clock sync and server tick broadcast. Snapshot history for rewind and lag compensation. NAT Traversal ICE with STUN and TURN. Prefer peer to server; peer to peer optional for niche modes with relays. Security TLS one point three for control streams and QUIC AEAD ciphers for datagrams. Token binding and replay protection. Input signing from anti-tamper SDK when available. Moderation and Abuse Rate limits per user, device, IP, ASN, and tenant. Server-side sanity checks for movement and actions. Observability Per packet RTT, jitter, loss, reordering, ECN, send queue depth, and tick time exported to telemetry.

Non

Availability ninety nine point nine five percent for control plane. Data plane ninety nine point nine percent per region. Latency p95 input to server receive under thirty five milliseconds in-region. Server tick under sixteen milliseconds at sixty Hertz or under eight milliseconds at one hundred twenty Hertz. Snapshot delivery under fifty milliseconds. Durability RPO zero for session and match outcomes. RTO under fifteen minutes per region. Scalability linear by active sessions and bandwidth with cost-aware autoscaling.

Architecture Overview

Cell Based Each region hosts Gateways for QUIC UDP and Websocket, Relays with Anycast, Matchmaking and Session services, Authoritative Game Servers, Telemetry Ingest, and Storage. A thin global control plane manages identity, configuration, policies, and fleet orchestration. Data Plane Client connects via QUIC UDP to nearest healthy gateway or relay. Control uses QUIC streams reliable and ordered. Game state uses datagrams unreliable and sequenced with selective reliability. Voice uses separate SRTP or WebRTC media path when enabled. Control Plane HTTPS APIs for matchmaking, session tokens, identity, inventory, and telemetry upload. All requests authenticated with OAuth two point one or OpenID Connect tokens and rate limited.

Data Flow Connect Client performs ICE. Gateway authenticates token, negotiates protocol parameters congestion profile, ack strategy, reliability classes, and encryption keys. Clock sync performed and

baseline snapshot delivered. Loop Client sends inputs with monotonic sequence and client time. Server simulates at fixed tick and emits delta snapshots and events using interest filters. Client predicts and then reconciles on authoritative snapshots with interpolation. Persistence Ranked outcomes, inventory changes, and ban or penalty decisions are persisted via outbox to durable stores; replays and summaries are appended to the lake.

Technology Stack Reference Transport and Relay QUIC over UDP with datagrams and streams. TURN and STUN for traversal. Anycast IPs for relays with scrubbing for DDoS. Websockets fallback. Gateways and Services Kubernetes with CNI tuned for UDP, API Gateway, service mesh with mTLS for internal calls. Telemetry and Streaming Kafka for metrics and logs, Flink for enrichment, ClickHouse for near real-time KPIs. Storage Postgres or Aurora for accounts and results, DynamoDB or Scylla for sessions and tokens, S3 or GCS with Iceberg or Delta for replays and telemetry. Observability OpenTelemetry, Prometheus and Grafana, Loki, Jaeger or Tempo; synthetic QoS probes per region.

Design Patterns and Protocol Choices Reliability Classes Reliable ordered for session control and inventory. Reliable unordered for chat and events that must arrive but can be out of order. Unreliable sequenced for state updates where only latest matters. Fire and forget for high-rate hints like input samples. Congestion Control Use BBR like pacing profile for variable last mile. Apply packet pacing and limit inflight datagrams by RTT and loss. Respect ECN when present. Fall back to Cubic for conservative paths. Disable Nagle and enable low latency socket options. Loss Recovery Ack ranges with selective retransmission for reliable channels. Redundant key frames at low rate for state. Forward error concealment on client to mask missing frames. Optional lightweight FEC for tournament modes. Framing and Compression Compact header with varints. Delta compression against baseline, dictionary compression for component names, optional zstd for key frames. Interest Management Spatial or logical partitioning. Budgeted sends per client and per tick. Suppress far or irrelevant entities. Drop cosmetics first under pressure. Clock Sync and Rewind Hybrid scheme using smoothed RTT to estimate offset. Maintain snapshot ring buffer for rewind and lag compensation with strict bounds. QoS and Placement QoS probes collect RTT and jitter per region. Matchmaker selects region considering load and fairness. Tokens encode placement to prevent spoofing. Security Controls Short-lived tokens bound to device and connection parameters. Anti replay windows. Input sanity checks and server authority for outcomes. Payload limits and schema validation.

Storage Strategy

Hot Path In process memory for simulation and snapshot buffers. Redis for session presence and rate limits with short TTL. No SQL KV for session tokens with conditional writes. Warm Path Relational store for outcomes and leaderboards with outbox to Kafka. No SQL for high-QPS counters. Cold Path Object storage for replays and telemetry in Parquet with Iceberg tables. ClickHouse for analytics and regional dashboards.

Scaling Approach Shard by region, tenant, mode, and possibly map. One process per match or small batch of matches. Gateways autoscale on active connections and p95 latency. Relays scale with bandwidth headroom and flow count. Pre-warm capacity for events and drops. Backpressure and Degradation Clamp send rate when loss spikes. Increase interpolation buffer modestly on jitter. Reduce update rate for distant entities and drop cosmetic streams first. Switch to reliable control only during severe congestion.

Data Models

Abridged Session session_id, player_id, region, gateway_id, relay_id, negotiated_profile, issued_at, expires_at, reconnect_token, last_seen_at. QoSProbe player_id, probe_id, region, rtt_ms, jitter_ms, loss_pct, timestamp. Input match_id, player_id, seq, client_time, command, payload, sent_at. Snapshot match_id, tick, server_time, baseline_id, delta_bytes, reliability_flag. Metrics session_id, rtt_ms, jitter_ms, loss_pct, reorder_pct, ecn, send_queue_depth, tick_cpu_ms, egress_kbps.

Latency Budgets and SLAs

Edge and auth under fifteen milliseconds p95. Input to server receive under thirty five milliseconds p95 in-region. Server tick under sixteen milliseconds at sixty Hertz. Snapshot send to client receive under fifty milliseconds p95. Perceived input to action under one hundred milliseconds p95 for fast-action. Availability control plane ninety nine point nine five percent. Data plane ninety nine point nine percent.

Security Reliability and Compliance PII minimization and pseudonymous ids in the data plane. TLS one point three everywhere. QUIC key rotation. RBAC and ABAC for admin APIs. WAF and bot mitigation at edge. DDoS aware relays and rate limits. GDPR and CCPA with data subject rights, retention, and regional residency. SOC two Type two evidence collection with audit trails.

Multi Tenancy Tenant id on every key and topic. Quotas and rate limits per tenant. Optional hard isolation with dedicated cells and VPCs. Per tenant encryption keys and dashboards.

CI CD and Release Safety Everything as code using Terraform or Pulumi and Helm. SAST DAST and IAST in pipeline. Signed images and SBOM with cosign. Canary rollouts by percentage of matches and regions with automatic rollback on SLO burn such as p95 spike or tick overrun. Schema registry and contract tests for protocols.

APIs and Integration Points Client POST find_match, POST join, GET qos_probes, POST reconnect, GET relay_token. Server POST report_result, POST upload_replay, POST telemetry_batch. Admin POST drain_server, POST toggle_canary, GET fleet_health, POST penalize_player. Integrations Identity, anti cheat, inventory, commerce, analytics export.

Observability and Operations

Golden signals RTT, jitter, loss, reorder, tick duration, send queue depth, snapshot size, disconnect rate, placement latency. Dashboards and alerts Error budget burn, tick overruns, packet loss spikes, region QoS degradation, canary regression. Probes Synthetic clients per region with active tests for UDP reachability and QUIC handshake success. Runbooks Relay saturation, broker lag, gateway overload, tick spike, bad canary, regional failover.

Capacity Planning and Cost Estimate Illustrative per busy region Concurrent players one hundred thousand to one million depending on title. Average egress forty to sixty kilobits per second per fast-action player. Gateways sized by connections and crypto offload. Relays provisioned with two times peak bandwidth headroom. Kafka six to nine brokers replication factor three for telemetry. Monthly cost order of magnitude thirty thousand to one hundred twenty thousand depending on concurrency and egress.

Testing Strategy and Failure Drills

Determinism and unit tests for sim and serialization. Property tests for fragmentation and reassembly, ack handling, and loss recovery. CI network emulation for latency, jitter, loss, and reordering. Load and soak tests with synthetic clients. Chaos drills for relay loss, gateway crash, broker backlog, and regional isolation. Disaster recovery drills for outcomes and replays integrity.

Phased Build Plan

MVP three months Single region cell, QUIC UDP transport with websocket fallback, basic matchmaking and session, reliable control plus unreliable state, prediction and reconciliation, telemetry MVP. Phase two six to nine months Multi region cells with Anycast relays, lag compensation and snapshot history, interest management, stronger congestion control, autoscaling, analytics. Phase three twelve months Hard isolation options per tenant, advanced FEC, regional failover playbooks, cost optimizations, stricter SLOs and capacity for tournaments.

Key Choices Trade Offs Risks and Mitigations QUIC over UDP reduces handshake and allows mixed reliability but adds complexity; mitigate with comprehensive test harnesses and fallbacks. Higher tick rates improve feel but raise CPU and bandwidth; expose per mode profiles and dynamic throttling. Aggressive prediction hides latency but increases artifacts; bound extrapolation and tune buffers. Relays cut RTT and bypass NATs but increase cost; deploy in problem regions first and scale with demand. Lag compensation

helps fairness but can be abused; clamp rewind and audit anomalies.